

Righteous IT

Join the crusade!

Linux Investigation (Part 4)

Posted on ~~May 7, 2026~~ May 4, 2026 by Hal Pomeranz

In this installment I'm going to tell you the sequence of steps I took to create the scenario data. If you want to read about my investigation before you look at "the answers", then please read Part 1

(<https://righteousit.com/2026/05/04/linux-investigation-part-1/>), Part 2

(<https://righteousit.com/2026/05/05/linux-investigation-part-2/>), and Part 3

(<https://righteousit.com/2026/05/06/linux-investigation-part-3/>).

Our scenario begins with the "worker" login at 23:22:19. Any artifacts before this time are my earlier attempts to get the scenario running. For those of you contemplating making your own forensic scenarios in the future, I suggest you carefully script the scenario and practice several times on a test system so that you don't make big mistakes and leave confusing artifacts like I did. Apologies again to those of you who thought the earlier artifacts needed to be explained as part of the scenario.

At 23:22:19 I logged into the system as user "worker" using the password for the account, which was "worker". The idea I had was to mimic investigations I've been involved in recently where the attackers accessed the system via stolen credentials. The frustrating thing about these incidents is that there are generally no artifacts showing how the attackers obtained the credentials unless you can find signs of an earlier exploit. Welcome to my world.

The "worker" had unlimited Sudo access and even had the "NOPASSWD" option set. Privilege escalation was trivial. Again, this mimics my real world experience in some cases.

One of the open questions from the investigation is how the `/dev/shm/kit` directory was staged, though we suspected the SSH session that occurred at 23:23:48. This was in fact me using `scp` to copy a file called `k.tgz` into `/dev/shm`. I then unpacked this file, creating `/dev/shm/kit`, and then removed the tar file.

The `/dev/shm/kit` directory contained three files:

- `rk.so` – the Father rootkit library
- `xmrig` – the XMRig cryptocurrency miner
- `config` – an SSH client config file, containing the parameters for the outbound connection

I regret that my earlier failed attempts at the scenario updated the atimes on the `scp` and `tar` binaries. Because of the "relative atime" updates in Linux, the fact that the atimes on these programs had been updated only a few hours earlier meant that they didn't get an update when I ran them during the scenario. If they had been updated during the scenario window, there would have been a much clearer answer to how this information got staged.

I had also hoped that we would be able to acquire the files in the directory via Volatility—particularly the "config" file—even though they had been deleted. Alas, this didn't work for me. I'll need to research a way to get them to hang around in memory for future versions.

Once `/dev/shm/kit` had been created from the tar file, I copied `rk.so` to `/usr/lib/x86_64-linux-gnu/libymv.so.3`. I then used `echo /usr/lib/x86_64-linux-gnu/libymv.so.3 >/etc/ld.so.preload` to create the `ld.so.preload` file. But I needed the SSH daemon to pick up the rootkit library, so I did `systemctl restart sshd`.

With the Father rootkit active in the SSH daemon, I went old school and connected to port 22 with `nc -p 48411 192.168.4.22 22`. I'd hard-coded `ymv` as the rootkit password when I compiled the rootkit. I then used the usual Python `pty.spawn()` trick (<https://blog.ropnop.com/upgrading-simple-shells-to-fully-interactive-ttys/>) to get a nice interactive shell— and create a clear artifact for the SOC to key in on. Once my backdoor shell was fully operational, I closed my original SSH session.

We found the command history for the hidden bash shell. You can see me renaming `xmrig` to `top` and setting up my search path so that I could execute `top` from the `/dev/shm/kit` directory. I meant to include an `export HISTFILE=/dev/null` command to give a clue about the lack of shell history on the system, but failed to do so.

The next step in the backdoor session was to fire up the outbound SSH session. For those of you who were wondering, here are the contents of the `config` file I used:

```
host ymv
Hostname 192.168.5.95
User pocmp
UserKnownHostsFile /dev/null
StrictHostKeyChecking no
LocalForward 3333 127.0.0.1:3333
SessionType none
StdinNull yes
RequestTTY no
ForwardX11 no
```

Maybe in the future I'll add the evidence from the outbound host to the scenario. That would feel more real-world.

After backgrounding the outbound SSH process, it was time to run our renamed XMRig. I hadn't considered the fact that the `-B` option would fully daemonize this process so that it inherited PPID 1. Frankly this was a happy accident from my perspective since it meant that our hidden `top` process was not immediately evident in the `linux.pstree` output.

With the cryptominer running, I popped up one level to `/dev/shm` (`cd ..`) and removed the `kit` directory (`rm -rf kit`). I debated using `shred` just to be evil, but decided it was too mean. Also I rarely see actual attackers using `shred`.

At this point I left the backdoor session running because I wanted the command history to be an artifact in the scenario. I logged in again as user `worker` and pretended to be the SOC running UAC to collect data from the system. Did anybody notice that the memory dump provided was not actually created by UAC? When I looked at the UAC dump, the memory file could not be parsed for even simple plugins. So I went back and ran AVML manually. Thankfully, I got a decent memory capture on the second try.

Personally I learned a lot creating this scenario and later analyzing it. I've got lots of ideas and notes for creating more such scenarios in the future. But wow is it a lot of work, so don't get too antsy while waiting for the next installment.

Posted in [DFIR](#), [Forensics](#), [Linux](#) Tagged [Contest](#), [DFIR](#), [Forensics](#), [Linux](#), [Scenario](#)



Published by Hal Pomeranz

Independent Computer Forensics and Information Security consultant. Expert Witness. Trainer. [View all posts by Hal Pomeranz](#)

[Blog at WordPress.com.](#)